ExamPredict AI : construire le cerveau

Le guide du backend. Des lycéens déposent les PDF des sujets des années passées (2, 20 ou 50), l'IA repère les thèmes qui reviennent, s'entraîne à prédire, puis génère un sujet probable pour le jour J. Chaque étape contient une explication simple, un exercice à coder toi-même et un test pour vérifier.

1. L'idée, expliquée à un enfant
2. Le plan de la machine
3. Étape 1 · Lire les PDF
4. Étape 2 · Découper en exercices
5. Étape 3 · Reconnaître les thèmes
6. Étape 4 · Ranger par année
7. Étape 5 · Le score de prédiction
8. Étape 6 · S'entraîner et se noter
9. Étape 7 · Générer le sujet
10. Étape 8 · Ouvrir la machine au monde (API)
11. La suite : vraie IA, site et comptes
12. À savoir par cœur

---

1. L'idée, expliquée à un enfant

Imagine un élève très appliqué. Avant le bac, il prend **tous les anciens sujets** et il les lit. À chaque exercice, il note dans un cahier : « 2024 : forces, ondes, chimie. 2025 : énergie, ondes, acide-base… ».

Au bout d'un moment, il remarque des choses : « les **ondes** tombent presque chaque année », « l'**optique** n'est pas tombée depuis 3 ans ». Alors il fait un pari : « cette année, je pense que ce sera ondes, énergie et optique ». Et il fabrique un **sujet d'entraînement** avec ces thèmes.

ExamPredict AI fait exactement ça, mais en quelques secondes, sans se fatiguer, et sur 50 sujets s'il le faut.

> **Définition 1.1** *(modèle)*
>
> Un **modèle** est une recette qui transforme des **données d'entrée** (les anciens sujets) en une **prédiction** (les thèmes probables). Il a des **réglages**, et l'**entraînement** consiste à trouver les réglages qui donnent les meilleures prédictions sur le passé.

> **Honnêteté avant tout**
>
> Personne ne peut **connaître** le sujet du bac à l'avance. ExamPredict donne des **probabilités** basées sur le passé. Un bon ML Engineer **mesure** la fiabilité de son modèle et l'**affiche** aux utilisateurs, par exemple : « sur les 5 dernières années, 3 thèmes sur 4 prédits sont vraiment tombés ». C'est ce qui rend un produit sérieux et digne de confiance.

2. Le plan de la machine

Une IA, c'est une **chaîne de petites machines**. Chacune fait une seule chose, puis passe le résultat à la suivante. On appelle ça un **pipeline** (un « tuyau »).

```
📄 PDF déposés
    │
    ▼
① LIRE           PDF → texte brut                    document_loader.py
    │
    ▼
② DÉCOUPER       texte → liste d'exercices           splitter.py
    │
    ▼
③ RECONNAÎTRE    exercice → thèmes {"ondes", …}      analyzer.py
    │
    ▼
④ RANGER         année → thèmes de l'année           history.py
    │
    ▼
⑤ PRÉDIRE        historique → score de chaque thème  predictor.py
    │
    ▼
⑥ S'ENTRAÎNER    cacher une année, prédire, comparer trainer.py
    │
    ▼
⑦ GÉNÉRER        top thèmes → sujet du jour J        generator.py
    │
    ▼
⑧ API            le site web parle à la machine      api.py
```

> **Règle d'architecture**
>
> **Un fichier = une responsabilité.** Si demain on remplace l'étape ③ par une vraie IA, on change **un seul fichier**, et tout le reste continue de fonctionner. C'est ce qu'on appelle un **module**.

L'arborescence cible du dépôt GitHub

```
exampredict-ai/
├── data/
│   └── exams/              # les PDF : 2024-Metro-J1-Sujet-Original.pdf…
├── src/
│   ├── document_loader.py  # ① existe déjà
│   ├── splitter.py         # ② à créer
│   ├── analyzer.py         # ③ existe, à corriger
│   ├── history.py          # ④ à créer
│   ├── predictor.py        # ⑤ à réécrire
│   ├── trainer.py          # ⑥ à créer
│   ├── generator.py        # ⑦ à créer
│   ├── api.py              # ⑧ à créer
│   └── main.py             # lance tout en ligne de commande
├── tests/
│   └── test_*.py           # un fichier de tests par module
├── requirements.txt
├── .gitignore
└── README.md
```

Nommer les PDF : une convention qui rend tout possible

Tes fichiers s'appellent déjà `2024-Metro-J1-Sujet-Original.pdf`. L'**année est au début du nom**, donc le programme peut la lire automatiquement. On impose ce format à tous les dépôts : `ANNÉE-CENTRE-JOUR-….pdf`. En ML, la qualité des données compte plus que l'algorithme.

---

3. Étape 1 · Lire les PDF

Analogie : un PDF est une photo de texte bien rangée. `pypdf` est la loupe qui en extrait les lettres.

Le fichier `document_loader.py` existe déjà et fonctionne. Il lui manque une chose : **survivre à un PDF cassé**. Si un lycéen dépose un fichier corrompu, tout le programme plante aujourd'hui.

Exercice 1 · Robustesse

Dans `load_documents`, entoure la lecture d'un fichier par `try / except Exception`. En cas d'erreur, affiche `Fichier ignoré : nom (raison)` et passe au suivant.

```
try:
    text = read_pdf(path)
except Exception as erreur:
    print(f"Fichier ignoré : {path.name} ({erreur})")
    continue
```

Test : crée un faux PDF (un fichier texte renommé `faux.pdf`) dans `data/exams`. Le programme doit afficher le message et continuer avec les autres.

---

4. Étape 2 · Découper en exercices

Analogie : pour savoir quels thèmes tombent, on ne regarde pas le sujet entier d'un bloc. On le découpe en exercices, comme on découpe un gâteau en parts.

Définition 4.1 *(expression régulière)*

Une **expression régulière** (*regex*) est un motif de recherche. `r"EXERCICE\s+\d+"` veut dire : le mot EXERCICE, puis au moins un espace (`\s+`), puis un ou plusieurs chiffres (`\d+`).

Exercice 2 · Créer `src/splitter.py`

```
import re

def split_exercises(text: str) -> list[str]:
    """Découpe le texte d'un sujet en exercices."""
    morceaux = re.split(r"(?i)exercice\s+\d+", text)
    # TODO : enlever le 1er morceau (l'en-tête avant l'exercice 1)
    # TODO : enlever les morceaux vides ou presque vides (moins de 50 carac
    #        → utilise une compréhension de liste (fiche F2 !)
    ...
```

`(?i)` veut dire « ignore les majuscules ». `re.split` coupe le texte à chaque endroit où le motif apparaît.

Test :

```
texte = "En-tête du sujet EXERCICE 1 " + "a" * 60 + " Exercice 2 " + "b" *
assert len(split_exercises(texte)) == 2
```

---

5. Étape 3 · Reconnaître les thèmes

Analogie : on donne à l'IA une liste de mots-indices par thème. Si elle voit « longueur d'onde » ou « fréquence », elle coche « ondes ». C'est déjà ce que fait `analyzer.py` avec le dictionnaire `CONCEPTS`.

Deux bugs dans `find_concepts` (annoncés en F2)

Bug A : les sous-mots. `text.count("ph")` compte aussi le « ph » de « **ph**ysique » et de « gra**ph**ique ». « base » est trouvé dans « **base** de données ».

Bug B : le double comptage. Dans « force de frottement », les mots-clés « force », « frottement » et « force de frottement » sont **tous** comptés : une seule mention vaut 3 points.

Exercice 3 · Corriger le bug A avec `\b`

En regex, `\b` veut dire « frontière de mot » : le début ou la fin d'un mot. `r"\bph\b"` trouve « ph » tout seul, mais pas dans « physique ».

```
def count_keyword(text: str, keyword: str) -> int:
    motif = r"\b" + re.escape(keyword) + r"\b"
    return len(re.findall(motif, text))
```

`re.escape` protège les caractères spéciaux comme l'apostrophe de « loi d'ohm ». Remplace `text.count(keyword)` par `count_keyword(text, keyword)`.

Test : `assert count_keyword("la physique et le ph", "ph") == 1`

Exercice 4 · Changer de question (corrige le bug B)

Pour prédire, on n'a pas besoin de savoir **combien de fois** un mot apparaît. On veut savoir **si** un thème est présent dans un exercice. Écris :

```
def themes_of(exercise: str) -> set[str]:
    """Renvoie l'ensemble des thèmes présents dans un exercice."""
    # TODO : pour chaque thème de CONCEPTS, si AU MOINS UN mot-clé est trou
    # Indice : any(count_keyword(texte, k) > 0 for k in keywords)
```

Un `set` (fiche F1, section 8) ne contient jamais de doublons : le bug B disparaît.

---

6. Étape 4 · Ranger par année

Analogie : c'est le cahier de l'élève appliqué. Une page par année, et sur chaque page, les thèmes tombés.

Exercice 5 · Créer `src/history.py`

```
from collections import defaultdict

def year_from_filename(name: str) -> int:
    """'2024-Metro-J1-Sujet-Original.pdf' → 2024"""
    # TODO : name.split("-")[0], converti en int

def build_history(documents: dict[str, str]) -> dict[int, set[str]]:
    """{nom_fichier: texte} → {année: thèmes tombés cette année-là}"""
    history = defaultdict(set)
    # TODO : pour chaque document → année, découper en exercices,
    #        ajouter les thèmes de chaque exercice dans history[année]
    #        (indice : un set se complète avec .update(autre_set))
    return dict(history)
```

Résultat attendu (forme) : `{2024: {"ondes", "forces", …}, 2025: {…}, 2026: {…}}`

Plusieurs sujets la même année (Métropole J1, J2, Asie…) ? Ils s'ajoutent tous à la même année : c'est le rôle de `defaultdict(set)`.

---

7. Étape 5 · Le score de prédiction

Analogie : pour parier sur un thème, l'élève se pose trois questions. Est-il **souvent** tombé ? Est-il tombé **récemment** ? Et est-il **absent depuis longtemps** (on dit qu'il est « dû ») ? Chaque réponse rapporte des points.

Définition 7.1 *(les trois signaux, notés de 0 à 1)*

Fréquence : le nombre d'années où le thème est tombé, divisé par le nombre total d'années.
Récence : 1 s'il est tombé l'an dernier, 0,5 il y a deux ans, 0,25 il y a trois ans… (on divise par 2 à chaque année qui passe).
Retard : le nombre d'années depuis sa dernière apparition, divisé par le nombre total d'années.

Formule 7.2 *(score)*

```
score = w_freq × fréquence  +  w_rec × récence  +  w_ret × retard
```

`w_freq`, `w_rec` et `w_ret` sont les **poids** : l'importance donnée à chaque signal. Ce sont les **réglages du modèle**. On ne les choisit pas au hasard : on les fait **apprendre** à l'étape 6.

Exercice 6 · Réécrire `src/predictor.py`

```python
def score_themes(history: dict[int, set[str]],
                 weights: tuple[float, float, float]) -> list[tuple[str, fl
    """Renvoie [(thème, score), …] trié du plus probable au moins probable.
    w_freq, w_rec, w_ret = weights
    annees = sorted(history)
    # TODO : récupérer tous les thèmes vus (union des sets)
    # TODO : pour chaque thème, calculer les 3 signaux, puis le score
    # TODO : trier avec sorted(..., key=lambda p: p[1], reverse=True)
```

Cas limite obligatoire : si `history` est vide, la fonction renvoie `[]` au lieu de planter. C'est le bug `max()` de l'ancien `predictor.py`.

---

8. Étape 6 · S'entraîner et se noter

C'est **l'étape la plus importante**, et c'est elle qui fait de toi un ML Engineer.

Analogie : pour savoir si ton pari est bon, tu fais semblant d'être en 2025. Tu **caches** le sujet 2025, tu prédis avec 2024 et avant, puis tu regardes le vrai sujet 2025. Combien de tes thèmes sont vraiment tombés ? Ensuite, tu recommences en faisant semblant d'être en 2026.

Définition 8.1 *(backtesting)*

Le **backtesting** évalue un modèle en le faisant prédire des années passées, **sans jamais lui montrer la réponse** pendant la prédiction. Pour chaque année Y : on entraîne sur les années < Y, on prédit Y, on compare avec la réalité.

Définition 8.2 *(précision@k)*

La **précision@k** est la part des k thèmes prédits qui sont vraiment tombés. Si on prédit 4 thèmes et que 3 sont tombés, précision@4 = 3/4 = 0,75.

Définition 8.3 *(entraînement = recherche des meilleurs réglages)*

On essaie plusieurs combinaisons de poids, par exemple chaque poids dans `{0, 0.5, 1}`, soit 27 combinaisons. On garde celle qui obtient la **meilleure précision moyenne en backtesting**. Cette méthode s'appelle la **recherche sur grille** (*grid search*). Elle est utilisée partout en ML.

Exercice 7 · Créer `src/trainer.py`

```
from itertools import product
from predictor import score_themes

def precision_at_k(predits: list[str], reels: set[str]) -> float:
    # TODO : (nombre de prédits présents dans reels) / len(predits)

def backtest(history, weights, k=4) -> float:
    """Précision@k moyenne sur toutes les années prédictibles."""
    annees = sorted(history)
    scores = []
    for i in range(1, len(annees)):                # il faut au moins 1 an
        passe = {a: history[a] for a in annees[:i]} # on CACHE l'année anne
        top = [t for t, _ in score_themes(passe, weights)[:k]]
        scores.append(precision_at_k(top, history[annees[i]]))
    # TODO : renvoyer la moyenne (attention : liste vide si une seule année

def train(history, k=4):
    grille = [0, 0.5, 1]
    # TODO : pour chaque (wf, wr, wt) in product(grille, repeat=3) → backte
    # TODO : renvoyer les meilleurs poids ET leur précision
```

`product(grille, repeat=3)` génère toutes les combinaisons : (0,0,0), (0,0,0.5), … (1,1,1).

Deux pièges que tout ML Engineer doit connaître

La fuite de données (*data leakage*) : si l'année cachée se glisse dans l'entraînement, le modèle « triche » et paraît génial. Il sera nul sur le vrai bac. C'est pour ça que `passe` ne contient que `annees[:i]`.

Le surapprentissage (*overfitting*) : avec 3 sujets, les « meilleurs » poids peuvent être de la chance. Plus les lycéens déposent de sujets (toutes les années, tous les centres), plus le modèle devient fiable. **Avec 2 PDF, on peut prédire, mais on ne peut presque rien mesurer.** L'application doit le dire.

Exercice 8 · La baseline

Un modèle n'a de valeur que s'il bat une méthode simple. Calcule la précision@k d'une prédiction **au hasard** (`random.sample` parmi les thèmes connus), puis compare-la à celle de ton modèle entraîné. Si ton modèle ne bat pas le hasard, il ne sert à rien. Les entreprises posent cette question en entretien.

---

9. Étape 7 · Générer le sujet du jour J

Analogie : l'élève a ses thèmes favoris. Il ouvre ses anciens sujets, prend **le meilleur exercice de chaque thème** et les agrafe ensemble : voilà un sujet d'entraînement sur mesure.

Version 1 · Assemblage (aujourd'hui, sans IA générative)

```
def generate_exam(predictions, exercises_by_theme, n=3) -> str:
    """Assemble un sujet avec un exercice réel par thème prédit."""
    # TODO : pour les n premiers thèmes prédits :
    #   - prendre l'exercice le plus RÉCENT de ce thème
    #   - l'ajouter sous un titre "EXERCICE i — THÈME (probabilité estimée)
    # TODO : renvoyer le texte complet du sujet
```

Il faut donc garder, dans `history.py`, un second dictionnaire `{thème: [(année, texte_exercice), …]}`. C'est un `defaultdict(list)` (fiche F2, section 4).

> **Version 2 · IA générative (semaine prochaine)**
>
> On donne à un grand modèle de langage (Claude, via l'API Anthropic) les thèmes prédits **et** 2 ou 3 vrais exercices de ces thèmes comme exemples. On lui demande d'écrire un **nouvel** exercice dans le même style. Cette technique s'appelle le **RAG** (*Retrieval-Augmented Generation*) : on **retrouve** des documents, puis on **génère** à partir d'eux. C'est l'une des compétences les plus demandées pour un AI Engineer en 2027.

---

10. Étape 8 · Ouvrir la machine au monde (API)

Analogie : la machine est dans une cuisine. L'**API**, c'est le guichet du restaurant. Le site web (le client) passe une commande au guichet, et la cuisine renvoie le plat. Le client ne voit jamais la cuisine.

MÉTHODE	ADRESSE	CE QUE ÇA FAIT
`POST`	`/exams`	Déposer un ou plusieurs PDF
`POST`	`/train`	Entraîner le modèle, renvoie les poids et la précision
`GET`	`/predictions`	Thèmes probables avec leur score
`GET`	`/exam`	Le sujet généré pour le jour J

On utilisera **FastAPI**, la bibliothèque Python la plus utilisée pour servir des modèles ML. `GET` sert à demander une information, `POST` à envoyer quelque chose. Cette étape sera faite **après** les étapes 1 à 7, quand le cerveau marchera en ligne de commande.

11. La suite : vraie IA, site et comptes

VERSION	CONTENU	COMPÉTENCE AI/ML 2027
v1 · aujourd'hui	Étapes 1 à 7 en ligne de commande, tests, commit GitHub	Python, pipeline, évaluation honnête
v2	Thèmes découverts automatiquement, sans dictionnaire écrit à la main : TF-IDF + clustering (scikit-learn)	ML non supervisé, vectorisation de texte
v3	Génération de nouveaux exercices avec Claude (RAG)	LLM, prompt engineering, RAG
v4	API FastAPI + base de données des sujets	Mise en production d'un modèle (MLOps)
v5	Site web : dépôt des PDF, comptes élèves, interface animée	Produit complet, déploiement

> **Pourquoi cet ordre**
>
> Un beau site sur un cerveau qui ne marche pas, c'est une coquille vide. Les recruteurs ML regardent d'abord le **backend**, les **tests** et l'**évaluation**. Le site viendra brancher l'API de l'étape 8 sans rien changer au cerveau. C'est tout l'intérêt des modules.

12. À savoir par cœur

1. Un **modèle** transforme des données en prédiction. L'**entraînement** cherche ses meilleurs réglages.
2. Un **pipeline** est une chaîne de modules : lire → découper → reconnaître → ranger → prédire → évaluer → générer.
3. Un fichier = une responsabilité. On peut changer un module sans casser les autres.
4. La regex `\b` marque une frontière de mot : elle évite de trouver « ph » dans « physique ».
5. On utilise un `set` quand on veut savoir **si** quelque chose est présent, sans compter les doublons.

6	Le **backtesting** : cacher une année, prédire, comparer. Jamais de réponse visible pendant la prédiction.
7	La **précision@k** est la part des k prédictions qui sont justes.
8	La **recherche sur grille** teste toutes les combinaisons de réglages et garde la meilleure.
9	Un modèle doit battre une **baseline** (le hasard), sinon il ne sert à rien.
10	Peu de données = beaucoup de doute. Un produit honnête affiche sa fiabilité mesurée.
